Lecture 7

Second order optimization algorithms

Recall:

For $f: \mathbb{R}^n \to \mathbb{R}$, the loss function

- Gradient: A vector whose components are partial derivatives in each directions

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}\right]^T \in \mathbb{R}^n$$

- Hessian is the gradient of the gradient
- **Quadratic approximation** of the function $f: \mathbb{R}^n \to \mathbb{R}$

$$H = \nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} & \cdots \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots \frac{\partial^2 f}{\partial x_n \partial x_n} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

For $f: \mathbb{R}^n \to \mathbb{R}$, the function approximation around x^* is

$$f(x) = f(x^*) + (x - x^*)\boldsymbol{\nabla f}(\boldsymbol{x^*}) + \frac{1}{2}(x - x^*)^T \boldsymbol{\nabla^2 f}(\boldsymbol{x^*})(x - x^*) + \cdots$$

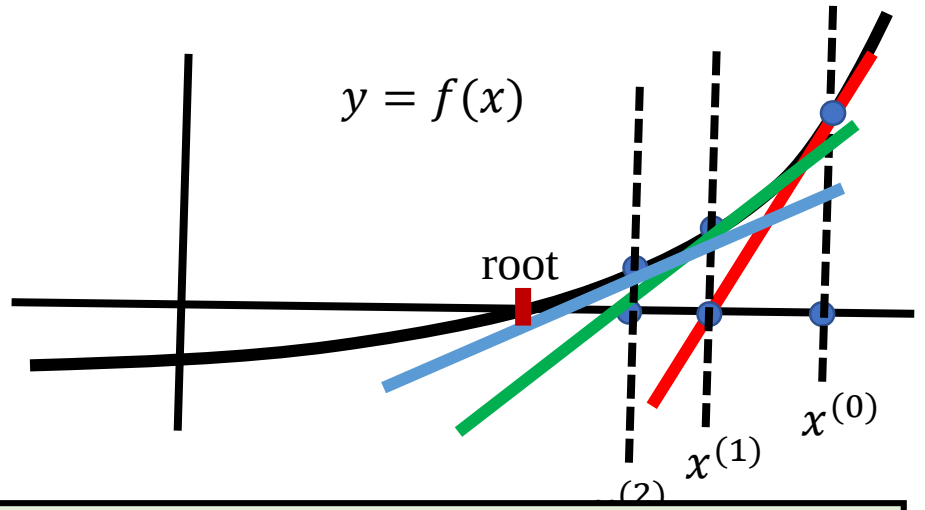
or (substitute $x = x^* + \alpha d$)

$$f(x^* + \alpha d) = f(x^*) + \boldsymbol{\alpha \nabla f}(\boldsymbol{x^*})\boldsymbol{d} + \frac{\alpha^2}{2} d^T \boldsymbol{\nabla^2 f}(\boldsymbol{x^*}) d + \cdots$$

Newton method for finding roots:

- Finding Roots (Zero Points) of $f(x)$ where $f: \mathbb{R} \to \mathbb{R}$
- To find a root of the function $f(x)$ (where $f(x)=0$), Newton's method iteratively updates x as follows:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$



- At any specific point x^*, the first order approximation of $f(x)$ is:

$$f(x) \approx f(x^*) + (x - x^*)f'(x^*)$$

- The aim is to find x such that $f(x) = 0$, for this we can use the linear approximation for $f(x)$ to solve for x:

$$\Rightarrow f(x^*) + (x - x^*)f'(x^*) = 0$$
$$\Rightarrow f(x^*) + xf'(x^*) - x^*f'(x^*) = 0$$
$$\Rightarrow x = x^* - \frac{f(x^*)}{f'(x^*)}$$

- $\frac{f(x^*)}{f'(x^*)} = (x^* - x)$ provides an estimate of how far x^* is from the root x, assuming $f(x)$ behaves like a straight line.
  - If $f(x^*)$ is large and $f'(x^*)$ is small, $\frac{f(x^*)}{f'(x^*)}$ is large, suggesting that x^* is relatively far from the root.
- Finally, denoting the current estimate x^* as $x^{(k)}$, the iterative formula is: $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$

Newton method for finding optima:

1

- Finding optima of $f(x)$ where $f: \mathbb{R} \to \mathbb{R}$ $(1D)$
- To find the optima (minima or maxima) of a function $f(x)$ in $1D$, Newton's method is applied to the derivative $f'(x)$ where the goal is to find the zero of $f'(x)$ (where $f'(x) = 0$).
- The update formula becomes:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

- At any specific point x^*, the second order approximation of $f(x)$ is:

$$f(x) \approx f(x^*) + (x - x^*)f'(x^*) + \frac{(x - x^*)^2}{2!} f''(x^*)$$

- Differentiate w r t x:

$$\Rightarrow \frac{d}{dx}(f(x)) \approx 0 + f'(x^*) + (x - x^*)f''(x^*)$$
$$\Rightarrow f'(x) \approx f'(x^*) + (x - x^*)f''(x^*)$$

- The goal is to find x such that $f'(x) = 0$

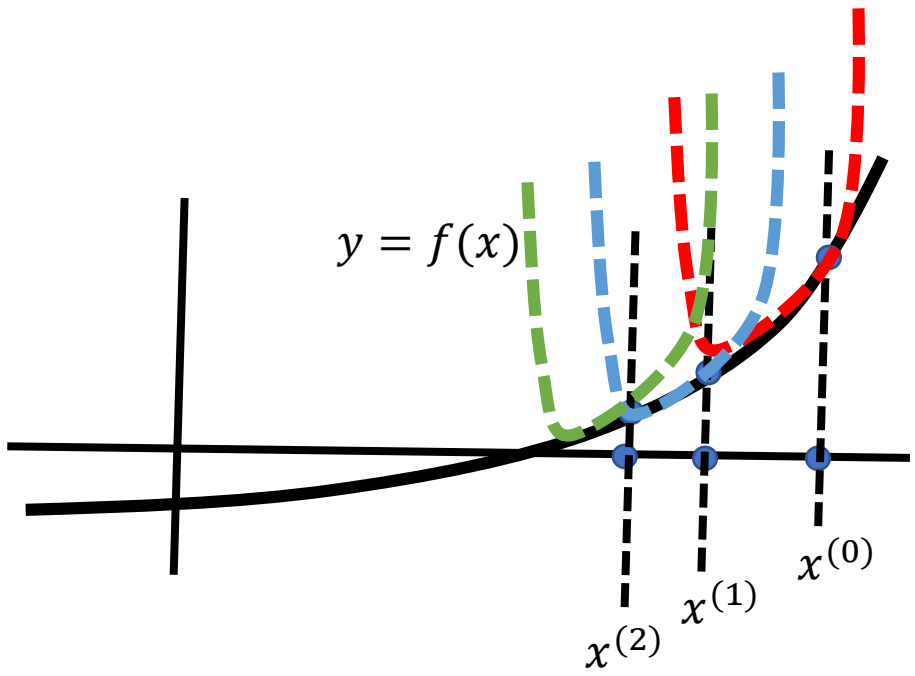
$$f'(x^*) + (x - x^*)f''(x^*) = 0$$

- Rearrange to Solve for x:

$$x = x^* - \frac{f'(x^*)}{f''(x^*)}$$

- Finally, denoting the current estimate x^* as $x^{(k)}$, the iterative formula is:

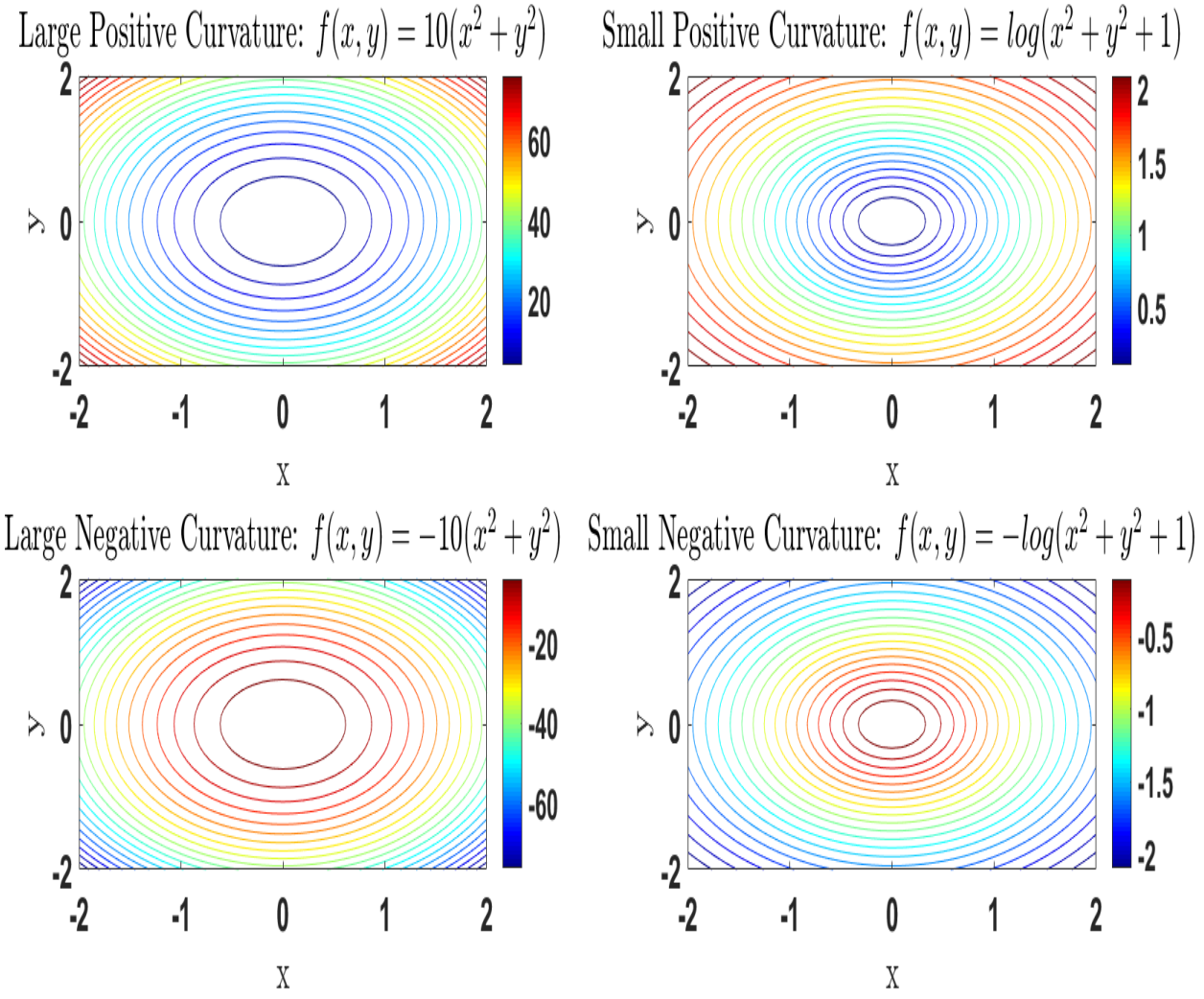
$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$



Newton method for finding optima:

- $f'(x^{(k)})$ represents slope of f at x^*
  - If $f'(x^{(k)}) > 0$, the f is increasing
  - If $f'(x^{(k)}) < 0$, the f is decreasing
  - **The value of $f'(x^{(k)})$ indicates how far we are from the optimum (where $f'(x) = 0$).**
  - Therefore, a larger magnitude of $f'(x^{(k)})$ suggests that we are further away from the optimal point.

- $f''(x^*)$: Behavior of the Rate of Change.
- measure the curvature of f at x^*.
- measures the rate of change of the first derivative $f'(x^*)$. Essentially, it tells us **how the slope of the function is changing at a point.**
- $f''(x^*) > 0 \Longrightarrow$ slope of f is increasing at x^*, f curves upwards, f curves upwards, f is convex (or concave up) at x^* or x^* is likely a local minimum.
- $f''(x^*) < 0 \Longrightarrow$ slope of f is decreasing at x^*, f is concave (or concave down) at x^* or x^* is likely a local maxima.
- $f''(x^*) = 0 \Longrightarrow$ saddle point
- **A large absolute value $f''(x^*)$ indicates that the slope is changing rapidly, it means the function is curving more steeply and therefore, small changes in x will lead to larger changes in $f(x)$**



Newton method for finding optima:

Newton method:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

- If $|f''(x^k)|$ is large: function is very sensitive to input changes.
  - The term $\frac{f'(x^{(k)})}{f''(x^{(k)})}$ will result in a **smaller adjustment** to $x^{(k)}$ because the denominator is large. This means that even if the slope $f'(x^{(k)})$ is significant, the adjustment will be controlled by the steepness of the curvature.

- If $|f''(x^k)|$ is small: The function may be nearly linear or have a flat minimum or maximum.
  - The term $\frac{f'(x^{(k)})}{f''(x^{(k)})}$ will result in a larger adjustment to $x^{(k)}$ because the denominator is small. This means that a relatively small slope could result in a large step towards the optimum if the curvature is not changing much.

Newton method for finding optima:

Newton method when $f: \mathbb{R} \to \mathbb{R}$ $(1D)$:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

The curvature plays a critical role in determining the adjustment step size in Newton's method:

- Larger $|f''(x^k)|$: Smaller controlled step sizes (more stability).
- Smaller $|f''(x^k)|$: Larger step sizes (more aggressive convergence).

Newton method when $f: \mathbb{R}^n \to \mathbb{R}$ (nD):

$$\boldsymbol{x^{(k+1)} = x^{(k)} - H^{-1}(x^{(k)})\nabla f\ (x^{(k)})}$$

- *Where* $H = \nabla^2 f(x^{(k)})$,
- $H^{-1}(x^{(k)})\nabla f\ (x^{(k)})$: $n \times 1$ *vector*

- *Each element* $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ *represents how the curvature in the* i^{th} *direction interacts with the* j^{th} *direction.*
- *The Hessian matrix encapsulates the* ***second-order information*** *about the function in all dimensions.*
- **Spectral Theorem — Diagonalization of** H
- Eigen decomposition: Any **symmetric matrix** H can be diagonalized:

$$H = Q\Lambda Q^T$$

  - $Q = [q_1, q_2, \dots, q_n] \in \mathbb{R}^{n \times n}$ is **orthogonal** matrix $(Q^T Q = I)$. Each q_i is **eigenvector of** $\boldsymbol{H}$
  - $\Lambda = \text{diag}\ (\lambda_1, \lambda_2, \dots, \lambda_n)$ is a diagonal matrix containing **eigenvalues** $\boldsymbol{\lambda_i}$ **corresponding to** $\boldsymbol{q_i}$

$$\boldsymbol{H q_i = \lambda_i q_i \Rightarrow H^{-1} q_i = \frac{1}{\lambda_i} q_i}$$

- **When the Hessian** $\boldsymbol{H}$ **operates on the direction** $\boldsymbol{q_i}$**, the result is the same direction** $\boldsymbol{q_i}$ **only scaled by** $\boldsymbol{\lambda_i}$

- 1

- *Since* $H = Q\Lambda Q^T$
- $H^{-1} = (Q\Lambda Q^T)^{-1} = (Q^T)^{-1}\Lambda^{-1}Q^{-1}$
- $H^{-1} = Q\Lambda^{-1}Q^T$, *where* $\Lambda^{-1} = \left(\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \ldots, \frac{1}{\lambda_n}\right)$ *and* $QQ^T = I \Rightarrow Q^{-1} = Q^T$
- *Express the gradient in the principal-direction basis:(linear combination of the basis vector)*

$$\Rightarrow \nabla f = \sum_{i=1}^{n} g_i q_i$$

$$\Rightarrow H^{-1}\,\nabla f \;\Rightarrow H^{-1}\left(\sum_{i=1}^{n} g_i q_i\right)$$

- *Because matrix multiplication is linear, we can distribute* H^{-1} *over the summation:*

$$\Rightarrow H^{-1}\,\nabla f \;\Rightarrow H^{-1}\left(\sum_{i=1}^{n} g_i q_i\right) \Rightarrow \sum_{i=1}^{n} g_i \boldsymbol{H^{-1}} \boldsymbol{q_i} \Rightarrow \sum_{i=1}^{n} g_i \left(\frac{\mathbf{1}}{\boldsymbol{\lambda_i}} \boldsymbol{q_i}\right) = \sum_{i=1}^{n} \frac{g_i}{\boldsymbol{\lambda_i}} q_i$$

- *Now position update:* $x^{(k+1)} = x^{(k)} - H^{-1}\left(x^{(k)}\right)\nabla f\,(x^{(k)})$

$$\Rightarrow x^{(k+1)} = x^{(k)} - \sum_{i=1}^{n} g_i \left(\frac{\mathbf{1}}{\boldsymbol{\lambda_i}} \boldsymbol{q_i}\right)$$

- *Large eigenvalues → steep directions → smaller steps.*
- *Small eigenvalues → flat directions → larger steps.*
  - $\boldsymbol{g_i}$ ***(gradient component)*** $\xrightarrow{\boldsymbol{H^{-1}}}$ $\frac{\boldsymbol{g_i}}{\boldsymbol{\lambda_i}}$

Positive definite hessian and stability

For stabilization:

$$\delta = -\left(\nabla^2 f\left(x^{(k)}\right)\right)^{-1} \nabla f\left(x^{(k)}\right)$$

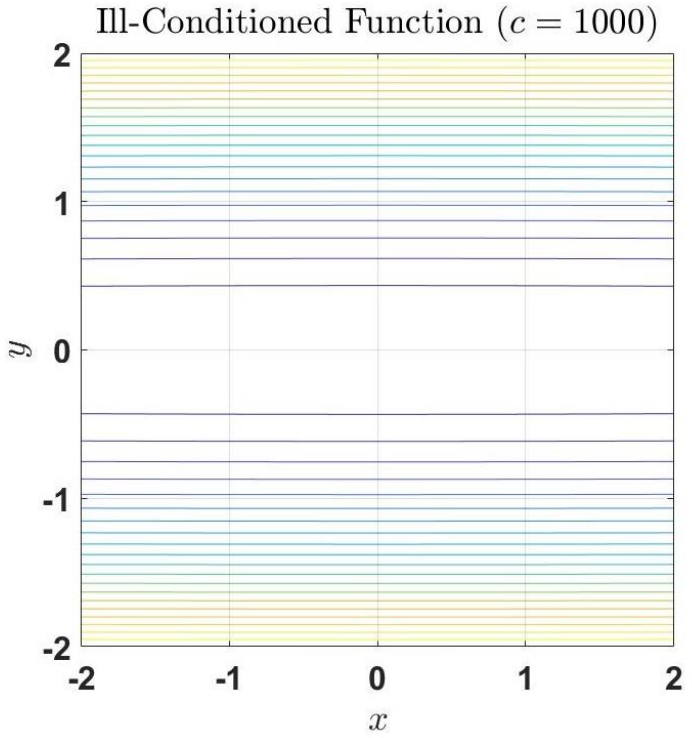
Where $\nabla^2 f\left(x^{(k)}\right)$ is the Hessian matrix, representing the curvature of f at $x^{(k)}$.

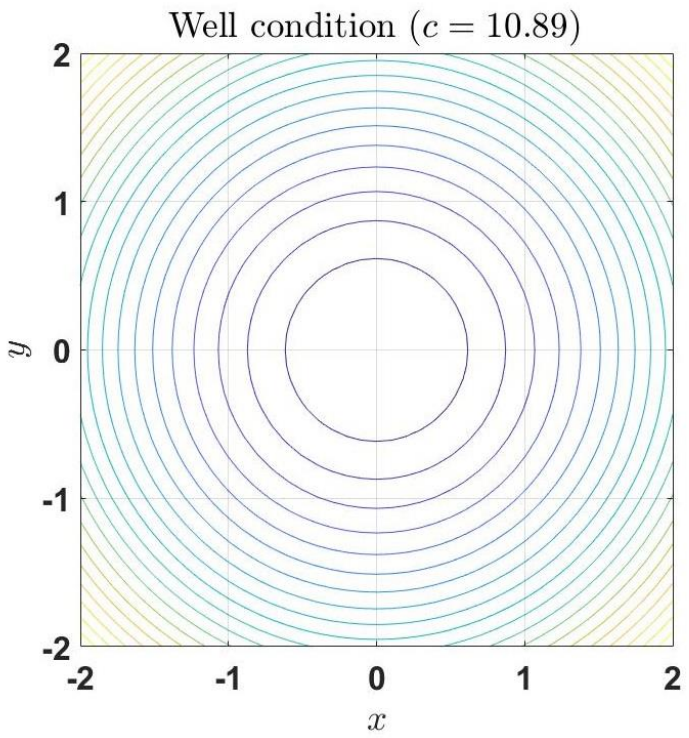
- If Hessian $\left(\nabla^2 f\left(x^{(k)}\right)\right)$ is **positive definite:**
  - $\left(\nabla^2 f\left(x^{(k)}\right)\right)$ has **positive eigenvalues.**
  - Geometrically, this means the function is **curved upward in all directions**: locally bowl shaped
  - In $-\left(\nabla^2 f\left(x^{(k)}\right)\right)^{-1} \nabla f\left(x^{(k)}\right)$,
    - Since the inverse of a positive definite matrix is also positive definite, -1 ensures that δ points downhill in all directions. The step δ always reduces the function value: **Decent direction**
  - The optimization is stable, and the algorithm moves safely toward a local minimum.
- If Hessian $\left(\nabla^2 f\left(x^{(k)}\right)\right)$ is not positive definite:
  - $\left(\nabla^2 f\left(x^{(k)}\right)\right)$ has negative eigenvalues.
  - Geometrically, this means the function is curved downward along some directions (concave in those directions).
  - In $-\left(\nabla^2 f\left(x^{(k)}\right)\right)^{-1} \nabla f\left(x^{(k)}\right)$, the negative eigenvalues can flip the direction, causing δ to **point uphill** instead of downhill.
  - If the step δ points uphill, the algorithm increases the function instead of decreasing it.
  - This can make the method unstable, and repeated steps may diverge away from the minimum rather than converge.

Damping effect:

- The damping factor λ is used to stabilize the solution process when the Hessian matrix is **ill-conditioned or singular.**
- By adding λ to the Hessian, we effectively modify its eigenvalues, which can improve numerical stability and influence the condition number of the modified Hessian.

- Let, $H = \begin{bmatrix} 1 & 0 \\ 0 & 0.001 \end{bmatrix}$
- $c(H) = \frac{\sigma_{max}}{\sigma_{mix}} = \frac{1}{0.001} = 1000$ it is ill-conditional Hessian
- Damping Hessian:
- $H_{damp} = H + \lambda \mathrm{I}$, let $\lambda = 0.1$
- $H_{damp} = \begin{bmatrix} 1 & 0 \\ 0 & 0.001 \end{bmatrix} + 0.1 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1.1 & 0 \\ 0 & 0.101 \end{bmatrix}$
- New condition number $c(H_{damp}) = \frac{1.1}{0.101} = 10.89$





Newton method for finding optima:

Newton method for function optimization

%Inputs
$f: \mathbb{R}^n \to \mathbb{R}$, a twice differentiable function
∇f: gradient of f, $\nabla^2 f$: PD Hessian of f
$x^{(0)}$: The initial guess for optimization solution
%Output: x^* a local minima of the loss function
Begin
    $k \leftarrow 0$
    While ($\| \nabla f(x^{(k)}) \| > \varepsilon$) **do**
        Compute the **Newton direction** $\delta^{(k)}$ by solving
        $\left(\nabla^2 f\left(x^{(k)}\right)\right)\delta^{(k)} = -\nabla f\left(x^{(k)}\right)$ % Newton system
        $x^{(k+1)} = x^{(k)} + \delta^{(k)}$
        $k = k + 1;$
    EndWhile
End

Damped Newton method for function optimization

%Inputs
$f: \mathbb{R}^n \to \mathbb{R}$, a twice differentiable function
∇f: gradient of f, $\nabla^2 f$: Hessian of f
λ: **Damping factor for the non-positive definite or non invertable Hessian**
$x^{(0)}$: The initial guess for optimization solution
%Output: x^* a local minima of the loss function
Begin
    $k \leftarrow 0$
    While ($\| \nabla f(x^{(k)}) \| > \varepsilon$) **do**
        Compute the **Newton direction** $\delta^{(k)}$ by solving
        $\left(\nabla^2 f\left(x^{(k)}\right) + \lambda I\right)\delta^{(k)} = -\nabla f\left(x^{(k)}\right)$
        % When $\nabla^2 f\left(x^{(k)}\right)$ is PD then Cholesky decomposition for Computing δ
        $x^{(k+1)} = x^{(k)} + \delta^{(k)}$
        $k = k + 1;$
    EndWhile
End

Damping factor λ:

For stabilization:

- When $\nabla^2 f\left(x^{(k)}\right)$ is not positive definite. The computed step δ could lead to **non-descent directions**, causing instability or divergence.

$$H_{damp} = (H + \lambda I)$$

- . If v_i is an eigenvector of H with eigenvalue λ_i, then for the modified Hessian H_{damp}:

$$H_{damp} v_i = (H + \lambda I) v_i = H v_i + \lambda I v_i$$
$$= \lambda_i v_i + \lambda I v_i$$
$$= (\lambda_i + \lambda) v_i$$

- Each eigenvalue is increased by λ.
- Adding λ shifts every Hessian eigenvalue upward by λ, while keeping the same eigenvectors.

Damping factor λ:

For stabilization:

- When $\nabla^2 f\left(x^{(k)}\right)$ is not positive definite. The computed step δ could lead to **non-descent directions**, causing instability or divergence.
- *Newton:*
  - $\left(\nabla^2 f\left(x^{(k)}\right)\right) \delta^{(k)} = -\nabla f\left(x^{(k)}\right)$
- *If* $\nabla^2 f\left(x^{(k)}\right)$ *is problematic:*
  - $\left(\nabla^2 f\left(x^{(k)}\right) + \lambda I\right) \delta^{(k)} = -\nabla f\left(x^{(k)}\right)$
- **In eigenvector basis:**
- $\boldsymbol{\delta^{(k)} = -\sum_{i=1}^{n} g_i \left(\frac{1}{\lambda_i + \lambda} q_i\right)}$
- *Damping shifts every Hessian eigenvalue from* λ_i *to* $\boldsymbol{\lambda_i + \lambda}$ *preventing very small or negative eigenvalues from producing problematic Newton steps.*

Example 1

Consider the function $f(x,y) = x^2 + 2xy + 3y^2 - 6x - 12y$
Use one iteration of Newton's method for minimization starting from the initial point $\left(x^{(0)}, y^{(0)}\right) = (0,0)^T$. What is the updated point $\left(x^{(1)}, y^{(1)}\right)^T$ after this iteration?

- $f(x,y) = x^2 + 2xy + 3y^2 - 6x - 12y$
- $\nabla f(x,y) = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix} = \begin{bmatrix} 2x - 4 \\ 6y + 6 \end{bmatrix}$
- $H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x \partial y \\ \partial^2 f/\partial y \partial x & \partial^2 f/\partial y^2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$
  - (Since the Hessian is constant here, it's the same at all points)
- Calculate $H^{-1} = \frac{1}{2\times 6 - 0\times 0} \begin{bmatrix} 6 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/6 \end{bmatrix}$

- Now Newton position update equation:
  - $\begin{bmatrix} x^{(k+1)} \\ y^{(k+1)} \end{bmatrix} = \begin{bmatrix} x^{(k)} \\ y^{(k)} \end{bmatrix} - H^{-1}(x^{(k)}, y^{(k)})\, \nabla f\left(x^{(k)}, y^{(k)}\right)$
- $\left(x^{(0)}, y^{(0)}\right) = (0,0)$
- First iteration $k = 0$:
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} x^{(0)} \\ y^{(0)} \end{bmatrix} - H^{-1}(x^{(0)}, y^{(0)})\, \nabla f\left(x^{(0)}, y^{(0)}\right)$

- Inverse of a matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$
- $A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

Example 1 contd.

- First iteration $k = 0$:
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} x^{(0)} \\ y^{(0)} \end{bmatrix} - H^{-1}(x^{(0)}, y^{(0)}) \nabla f(x^{(0)}, y^{(0)})$
  - $H^{-1}(x^{(k)}, y^{(k)}) = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/6 \end{bmatrix} \Rightarrow H^{-1}(0,0) = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/6 \end{bmatrix}$
  - $\nabla f(x^{(k)}, y^{(k)}) = \begin{bmatrix} 2x^{(0)} - 4 \\ 6y^{(0)} + 6 \end{bmatrix} \Rightarrow \nabla f(0,0) = \begin{bmatrix} 2.0 - 4 \\ 6.0 + 6 \end{bmatrix} = \begin{bmatrix} -4 \\ 6 \end{bmatrix}$
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{6} \end{bmatrix} \begin{bmatrix} -4 \\ 6 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$

- Second iteration $k = 1$:
- $\begin{bmatrix} x^{(2)} \\ y^{(2)} \end{bmatrix} = \begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} - H^{-1}(x^{(1)}, y^{(1)}) \nabla f(x^{(1)}, y^{(1)})$
  - $H^{-1}(x^{(k)}, y^{(k)}) = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/6 \end{bmatrix} \Rightarrow H^{-1}(2,-1) = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/6 \end{bmatrix}$
  - $\nabla f(x^{(k)}, y^{(k)}) = \begin{bmatrix} 2x^{(1)} - 4 \\ 6y^{(1)} + 6 \end{bmatrix} \Rightarrow \nabla f(2,-1) = \begin{bmatrix} 2.2 - 4 \\ 6.-1 + 6 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
  - It means $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$ is a stationary point. It means local minima
- $f(x^{(0)}, y^{(0)}) = f(0,0) = 0$
- $f(x^{(1)}, y^{(1)}) = f(2,-1) = -7$

Example 2

Consider the function $f(x,y) = x^2 + 2xy + y^2$
Use one iteration of Newton's method for minimization starting from the initial point $\left(x^{(0)}, y^{(0)}\right) = (1,0)^T$. What is the updated point $\left(x^{(1)}, y^{(1)}\right)^T$ after this iteration?

- $f(x,y) = x^2 + 2xy + y^2$
- $\nabla f(x,y) = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix} = \begin{bmatrix} 2x + 2y \\ 2x + 2y \end{bmatrix}$
- $H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x \partial y \\ \partial^2 f/\partial y \partial x & \partial^2 f/\partial y^2 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$ (eigenvalues $\lambda_1 = 4, \lambda_2 = 0$: positive semi definite hessian)
  - (Since the Hessian is constant here, it's the same at all points)
- Calculate H^{-1} =?
- Since Det$(H) = 2.2 - 2.2 = 0$, it means H is singular and does not have inverse. Here Newton method can not apply. Damped Newton method is useful these cases. Let $\lambda = 1$
- For damped Newton method, we need to calculate $(H + \lambda I)^{-1}$=?
  - $(H + \lambda I) = \begin{bmatrix} 2+1 & 2 \\ 2 & 2+1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$
  - $(H + \lambda I)^{-1} = \frac{1}{3.3-2.2} \begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 3/5 & -2/5 \\ -2/5 & 3/5 \end{bmatrix}$

Example 2 contd.

- Now Dampled Newton position update equation:
  - $\begin{bmatrix} x^{(k+1)} \\ y^{(k+1)} \end{bmatrix} = \begin{bmatrix} x^{(k)} \\ y^{(k)} \end{bmatrix} - (H + \lambda I)^{-1}(x^{(k)}, y^{(k)})\, \nabla f\left(x^{(k)}, y^{(k)}\right)$
- $\left(x^{(0)}, y^{(0)}\right) = (1,0)$
- First iteration $k = 0$:
  - $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} x^{(0)} \\ y^{(0)} \end{bmatrix} - (H + \lambda I)^{-1}(x^{(0)}, y^{(0)})\, \nabla f\left(x^{(0)}, y^{(0)}\right)$
  - $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 3/5 & -2/5 \\ -2/5 & 3/5 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
  - $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \times \frac{3}{5} - \frac{2}{5} \times 2 \\ 2 \times \left(-\frac{2}{5}\right) + 2 \times \frac{3}{5} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 0.4 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix}$
  - $f\left(x^{(0)}, y^{(0)}\right) = f(1,0) = 1$
  - $f\left(x^{(1)}, y^{(1)}\right) = f(0.6, -0.4) = 0.04$
- Second iterations continues…

Characteristics of Well-Conditioned Functions/ Newton method for well-condition functions

- A function is well-conditioned near a local minimum when its Hessian has **positive eigenvalues of comparable magnitude:**

$$0 < \lambda_{min} \approx \lambda_{max}$$

- Relatively uniform curvature
- Small condition number: $c(H) = \frac{\lambda_{max}}{\lambda_{min}}$

- Type equation here.Stable Newton direction:
- $x^{(k+1)} = x^{(k)} - H^{-1}\left(x^{(k)}\right)\nabla f\ (x^{(k)})$
- *Using eigenbasis:*
- $x^{(k+1)} = x^{(k)} - \sum_{i=1}^{n} g_i \left(\frac{1}{\lambda_i}\boldsymbol{q_i}\right)$
  - Comparable λ_i 's prevent the factors $\frac{1}{\lambda_i}$ from becoming excessively different, avoiding strong amplification or suppression of gradient components across principal directions.

Well-conditioned Hessian → relatively uniform curvature → balanced Newton steps → better numerical stability.

Quadratic function:

- A quadratic function is a polynomial function of degree 2, which means its highest power of the variable(s) is 2.
  - In one variable ($1D$): $f: \mathbb{R} \to \mathbb{R}$, the general quadratic form is: $f(x) = ax^2 + bx + c$, where $a, b, c \in \mathbb{R}$ and $a \neq 0$
  - In two variables ($2D$): $f: \mathbb{R}^2 \to \mathbb{R}$, the general quadratic form is:
    - $f(x_1, x_2) = a_{11}x_1^2 + a_{12}x_1x_2 + a_{13}x_2^2 + b_1x_1 + b_2x_2 + c$
    - $f(x_1, x_2) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a_{11} & \frac{a_{12}}{2} \\ \frac{a_{12}}{2} & a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + c$
    - $f(x_1, x_2) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \frac{1}{2} \begin{bmatrix} 2a_{11} & 2\frac{a_{12}}{2} \\ 2\frac{a_{12}}{2} & 2a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + c$
    - $f(x_1, x_2) = \frac{1}{2} \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2a_{11} & a_{12} \\ a_{12} & 2a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + c$
      - $\nabla f(x_1, x_2) = \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \end{bmatrix} = \begin{bmatrix} 2a_{11}x_1 + a_{12}x_2 + b_1 \\ 2a_{13}x_2 + a_{12}x_1 + b_2 \end{bmatrix} = \begin{bmatrix} a_{11} & \frac{a_{12}}{2} \\ \frac{a_{12}}{2} & a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2a_{11} & a_{12} \\ a_{12} & 2a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$
      - $H\big(f(x_1, x_2)\big) = \begin{bmatrix} \partial^2 f/\partial x_1^2 & \partial^2 f/\partial x_1 \partial x_2 \\ \partial^2 f/\partial x_2 \partial x_1 & \partial^2 f/\partial x_2^2 \end{bmatrix} = \begin{bmatrix} 2a_{11} & a_{12} \\ a_{12} & 2a_{13} \end{bmatrix}$ Constant!

Quadratic function: contd.

- In two variables ($2D$): $f: \mathbb{R}^2 \rightarrow \mathbb{R}$, the general quadratic form is:
  - $f(x_1, x_2) = a_{11}x_1^2 + a_{12}x_1x_2 + a_{13}x_2^2 + b_1x_1 + b_2x_2 + c$
  - $f(x_1, x_2) = \frac{1}{2}[x_1 \;\; x_2] \begin{bmatrix} 2a_{11} & a_{12} \\ a_{12} & 2a_{13} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [b_1 \;\; b_2] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + c = \frac{1}{2}X^THX + B^TX + c$
  - $f(x_1, x_2) = \frac{1}{2}X^THX + B^TX + c$, where $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$, c is a constant
  - $\nabla f(x_1, x_2) = \frac{1}{2}HX + b$

In n variables (nD): $f: \mathbb{R}^n \rightarrow \mathbb{R}$, the general quadratic form is:

- $f(X) = \frac{1}{2}X^THX + B^TX + c$, where $X \in \mathbb{R}^n, H \in \mathbb{R}^{n \times n}$ (H is Hessian matrix), $B \in \mathbb{R}^n, c \in \mathbb{R}$
- $\nabla f(X) = \frac{1}{2}HX + b$

- For a symmetric Hessian H, the quadratic form is

$$qr(d) = d^T H d$$

- Where $d \in \mathbb{R}^n$ is any direction.
- this quantity $qr(d)$ tells us how the objective function curves when we move in direction d

Quadratic Form and Principal Curvatures

- 1

- **Spectral Theorem — Diagonalization of H**
- Eigen decomposition: Any **symmetric matrix** H can be diagonalized:

$$H = Q\Lambda Q^T$$

  - $Q = [q_1, q_2, \dots, q_n] \in \mathbb{R}^{n\times n}$ is **orthogonal** matrix ($Q^TQ = I$). Each q_i is **eigenvector of H**
  - $\Lambda = \text{diag}\,(\lambda_1, \lambda_2, \dots, \lambda_n)$ is a diagonal matrix containing **eigenvalues λ_i corresponding to q_i**
    - $\boldsymbol{Hq_i = \lambda_i q_i}$

- *If $d = q_i$ (any Principal direction)*
- $Hq_i = \lambda_i q_i$
- $q_i^T H q_i = q_i^T(\lambda_i q_i) = \lambda_i q_i^T q_i$
- Because the eigenvectors are normalized, i.e., $q_i^T q_i$=1
  - $\boldsymbol{q_i^T H q_i = \lambda_i}$
- The eigenvalue λ_i is the curvature of the function in the principal direction q_i.

- If d is any arbitrary direction:
- (since $q_1, q_2, \dots, q_n$ is LI).
- Let

$$d = \sum_{i=1}^{n} a_i q_i \,, d^T = \sum_{i=1}^{n} a_i q_i^T$$

- Now, $d^T H d = \left(\sum_{i=1}^{n} a_i q_i^T\right) H \left(\sum_{i=1}^{n} a_i q_i\right)$
- $d^T H d = \sum_i \sum_j a_i a_j q_i^T H q_j = \sum_i \sum_j a_i a_j q_i^T \lambda_j q_j = \sum_i \sum_j a_i a_j \lambda_j q_i^T q_j$
- Since $q_i^T q_j = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$. Therefore, all cross terms disappears

$$d^T H d = \sum_{i=1}^{n} a_i^2 \ \boldsymbol{\lambda_i}$$

- An arbitrary direction is a combination of the Hessian's principal directions, and its curvature is the weighted sum of the corresponding eigenvalues, with weights a_i^2

Example

- A general quadratic function $f: \mathbb{R}^n \to \mathbb{R}$ defined as: $f(x) = \frac{1}{2}x^T Ax + b^T x + c$
  - $x \in \mathbb{R}^n$ is a variable vector
  - Where $A \in \mathbb{R}^{n \times n}$ is hessian of f (always symmetric $A^T = A$)
  - $b \in \mathbb{R}^n$ is a coefficient vector
  - $c \in \mathbb{R}$ is a constant
- The term $\frac{1}{2}x^T Ax$ is the quadratic term (e.g., $x_1^2, x_2^2, x_1 x_2, \ldots$) The term $b^T x$ is the linear term (e.g., $x_1, x_2, \ldots$)
- Let $A = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$, symmetric matrix, $b = \begin{bmatrix} -2 \\ -6 \end{bmatrix}, c = 5$ then quadratic form: $f(x) = \frac{1}{2}x^T \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} x + \begin{bmatrix} -2 & -6 \end{bmatrix}^T x + 5$
- Take $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, f(x_1, x_2) = \frac{1}{2}\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} -2 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 5 = \frac{3}{2}x_1^2 + x_1 x_2 + x_2^2 - 2x_1 - 6x_2 + 5$

One-Step Convergence of Newton's Method on Quadratic Functions

- Consider a **general quadratic function** $f: \mathbb{R}^n \to \mathbb{R}$ defined as: $f(x) = \frac{1}{2}x^T Ax + b^T x + c$
  - Where A is **positive definite or non singular** hessian of f
  - b is n dimensional vector
  - c is a constant
- Now gradient $\nabla f = Ax + b$
- Hessian $\nabla^2 f = A$
- Newton's Method updates the current estimate $x^{(k)}$ as follows:

$$x^{(k+1)} = x^{(k)} - \left(\nabla^2 f(x^{(k)})\right)^{-1} \nabla f(x^{(k)})$$
$$\Rightarrow x^{(k+1)} = x^{(k)} - (A)^{-1}(Ax^{(k)} + b)$$
$$\Rightarrow x^{(k+1)} = x^{(k)} - x^{(k)} - A^{-1}b$$
$$\Rightarrow x^{(k+1)} = -A^{-1}b$$

- The stationary point of the given function $f(x)$ is a point x where $\nabla f(x) = Ax + b = 0 \Rightarrow x = -A^{-1}b$
- Therefore, from the update equation: $x^{(k+1)} = -A^{-1}b$, we observe that $x^{(k+1)}$ is exactly the optimal point after the first iteration, **regardless of the initial point $x^{(0)}$**.
- Newton's Method finds the stationary point in **one step** because a quadratic function is **its own second-order approximation**. As long as the Hessian is invertible (the 'bowl' isn't perfectly flat in any direction), the method solves the linear gradient equation ($Ax + b = 0$) instantly.

Example 1

Consider the function $f(x,y) = 3x^2 + 2xy + 2y^2$
Use one iteration of Newton's method for minimization starting from the initial point $\left(x^{(0)}, y^{(0)}\right) = (1,1)^T$. What is the updated point $\left(x^{(1)}, y^{(1)}\right)^T$ after this iteration?

- $f(x,y) = 3x^2 + 2xy + 2y^2$
- $\nabla f(x,y) = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix} = \begin{bmatrix} 6x + 2y \\ 2x + 4y \end{bmatrix}$
- $H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x \partial y \\ \partial^2 f/\partial y \partial x & \partial^2 f/\partial y^2 \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ 2 & 4 \end{bmatrix}$ (eigenvalues $\lambda_1 = 5 + \sqrt{5}, \lambda_2 = 5 - \sqrt{5}$: positive definite hessian)
  - (Since the Hessian is constant here, it's the same at all points)
- Calculate H^{-1} =?
- Since Det$(H) = 6.4 - 2.2 = 20$, it means H is non-singular, so inverse exists
  - $(H)^{-1} = \frac{1}{6.4-2.2}\begin{bmatrix} 4 & -2 \\ -2 & 6 \end{bmatrix}$

- Inverse of a matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$
- $A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

Example 1 contd.

- Now Newton position update equation:
  - $\begin{bmatrix} x^{(k+1)} \\ y^{(k+1)} \end{bmatrix} = \begin{bmatrix} x^{(k)} \\ y^{(k)} \end{bmatrix} - H^{-1}(x^{(k)}, y^{(k)})\, \nabla f\left(x^{(k)}, y^{(k)}\right)$
- $\left(x^{(0)}, y^{(0)}\right) = (1,1)$
- $\nabla f\left(x^{(0)}, y^{(0)}\right) = \begin{bmatrix} 6x^{(0)} + 2y^{(0)} \\ 2x^{(0)} + 4y^{(0)} \end{bmatrix} \Rightarrow \nabla f(1,1) = \begin{bmatrix} 6 \times 1 + 2 \times 1 \\ 2 \times 1 + 4 \times 1 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$
- First iteration $k = 0$:
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} x^{(0)} \\ y^{(0)} \end{bmatrix} - H^{-1}(x^{(0)}, y^{(0)})\, \nabla f\left(x^{(0)}, y^{(0)}\right)$
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{6.4-2.2} \begin{bmatrix} 4 & -2 \\ -2 & 6 \end{bmatrix} \begin{bmatrix} 8 \\ 6 \end{bmatrix}$
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{20} \begin{bmatrix} 4 \times 8 - 2 \times 6 \\ -2 \times 8 + 6 \times 6 \end{bmatrix}$
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \frac{1}{20} \begin{bmatrix} 20 \\ 20 \end{bmatrix}$
- $\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

- Now $\nabla f\left(x^{(1)}, y^{(1)}\right) = \begin{bmatrix} 6 \times 0 + 2 \times 0 \\ 2 \times 0 + 4 \times 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Since the given function has PD Hessian and now $\nabla f\left(x^{(1)}, y^{(1)}\right) = (0\ \ 0)^T$, it means the point $\left(x^{(1)}, y^{(1)}\right)^T = (0\ \ 0)^T$ is the global minima

Quadratic function with newton method

Hessian	**Eigenvalues**	**One-step Newton?**	**Stationary point**
Positive definite $A \succ 0$	All $\lambda_i > 0$	**Yes**	Unique global **minimum**
Negative definite $A \prec 0$	All $\lambda_i < 0$	**Yes**	Unique global **maximum**
Indefinite, nonsingular	Some >0, some <0, none =0	**Yes**	**Saddle point**
Positive semidefinite	$\lambda_i > 0$, at least one =0	**No, standard Newton**	Hessian alone may be inconclusive
Negative semidefinite	λ_i<0, at least one =0	**No, standard Newton**	Hessian alone may be inconclusive
Indefinite, singular	Some >0, some <0, at least one =0	**No, standard Newton**	Saddle-type behavior; standard Newton undefined

Approximate Newton Methods

- **Newton method:**
- Advantages: fast convergence, robustness, affine invariance
- Disadvantages: requires second derivatives and solution of linear equation, can be too expensive for large scale applications

- In high-dimensional optimization problems, using exact Newton steps can be inefficient or even impractical due to the computational and storage challenges associated with the Hessian matrix, H, which is $n \times n$ in size.
- To address this, **approximate Newton methods** are employed to obtain the benefits of second-order optimization **without directly computing or storing the full Hessian**.

- Why Approximate Newton Methods?
  - **High Computational Cost**: Calculating and storing the dense Hessian for large n is computationally expensive and memory-intensive.
  - **Scalability Issues**: For large-scale problems, the time complexity for inverting the Hessian is $O(n^3)$ making it unsuitable for high-dimensional applications.
  - **Sparse Hessian Matrices**: Many large-scale problems have Hessians that are sparse, which means most of the elements are zero. Exploiting this sparsity can significantly reduce the computational burden.

Quasi newton methods

- Instead of computing the Hessian directly, **quasi-Newton methods** build up an approximation $B^{(k)}$ to the Hessian iteratively.

- Update Rule: The parameter update in quasi-Newton methods follows a similar structure to the Newton method:

$$x^{(k+1)} = x^{(k)} - \alpha^{(k)} B^{(k)^{-1}} \nabla f(x^{(k)})$$

  - where $\alpha^{(k)}$ is a step size (by Armijo rule), $\boldsymbol{B^{(k)}}$ **is the approximation of the Hessian**, and $\nabla f(x^{(k)})$ is the gradient at the current iteration.

- Quasi-Newton methods, like steepest descent, require only the gradient of the objective function.
- These methods update an **approximation of the Hessian matrix (or its inverse)** at each iteration by measuring how the ***gradient changes with respect to the changes in the input.***

- **Large-scale optimization** involves solving optimization problems that have a significant number of variables, constraints, or both. Characteristics: High Dimensionality, Complex Constraints, Non-convexity and Sparse data
- *Quasi-Newton methods strike a balance between the robustness and convergence speed of Newton's method and the computational efficiency of first-order methods.*

- Gradient: For $f: \mathbb{R}^n \to \mathbb{R}$, the loss function, a vector whose components are partial derivatives in each directions

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right]^T \in \mathbb{R}^n$$

- *Gradient: rate of the change of functions in all coordinate directions*
- *The gradient tells us how f changes.*
- ***But the gradient can itself change as x changes.***

- *A Jacobian is the matrix that represents the first-order rate of change of a vector-valued function with respect to its input variables.*
- ***Jacobian is the rate of change of vector valued function***
- *Let* $F: \mathbb{R}^n \to \mathbb{R}^m$ with $F(x) = [F_1(x), F_2(x), \ldots, F_m(x)]$ *where* $x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$
- $J_F(x) = \frac{\partial F}{\partial x} = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} \cdots \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} \cdots \frac{\partial F_2}{\partial x_n} \\ \vdots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \frac{\partial F_n}{\partial x_2} \cdots \frac{\partial F_n}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{n \times n}$

Gradient: rate of change of the function

Hessian : rate of change of the Gradient or Jacobian of the gradient.

- **Quadratic approximation** of the function $f: \mathbb{R}^n \to \mathbb{R}$
- $H(x) = \nabla^2 f(x) = J_{\nabla f}(x) = \frac{\partial \nabla f(x)}{\partial x} = \frac{\partial}{\partial x}\left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n}\right]^T$ *where* $x = (x_1, x_2, \dots, x_n)$
- *Take every component of the gradient and differentiate it with respect to every variable* $x_1, x_2, \dots, x_n$

$$H = J_{\nabla f}(x) = \nabla^2 f = \begin{bmatrix} \frac{\partial}{\partial x_1}\left(\frac{\partial f}{\partial x_1}\right) & \frac{\partial}{\partial x_2}\left(\frac{\partial f}{\partial x_1}\right) \dots \frac{\partial}{\partial x_n}\left(\frac{\partial f}{\partial x_1}\right) \\ \frac{\partial}{\partial x_2}\left(\frac{\partial f}{\partial x_2}\right) & \frac{\partial}{\partial x_2}\left(\frac{\partial f}{\partial x_2}\right) \dots \frac{\partial}{\partial x_n}\left(\frac{\partial f}{\partial x_2}\right) \\ \vdots & \vdots \\ \frac{\partial}{\partial x_n}\left(\frac{\partial f}{\partial x_n}\right) & \frac{\partial}{\partial x_2}\left(\frac{\partial f}{\partial x_n}\right) \dots \frac{\partial}{\partial x_n}\left(\frac{\partial f}{\partial x_n}\right) \end{bmatrix} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} & \dots \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots \frac{\partial^2 f}{\partial x_n \partial x_n} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Secant method for finding root: Recall

- Secant method is a numerical technique for finding root of $f(x) = 0$
- It uses two initial guesses (points) to iteratively refine the estimate of the root. **No derivative required.**
- Choose Initial Points: Select two points $x^{(k-1)}$ and $x^{(k)}$ such that $f(x^{(k-1)})$ and $f(x^{(k)})$ are known and $f(x^{(k)}) \neq f(x^{(k-1)})$. These points should be near the root.
- Compute the Secant Line with slope m:

$$m = \frac{f\left(x^{(k-1)}\right) - f(x^{(k)})}{x^{(k-1)} - x^{(k)}}$$

- ***This slope is a finite difference approximation of the derivative of the function $f(x)$ at a point between $x^{(k-1)}$ and $x^{(k)}$.***
- The secant line passing through $(x^{(k)}, f(x^{(k)}))$ with slope m is:

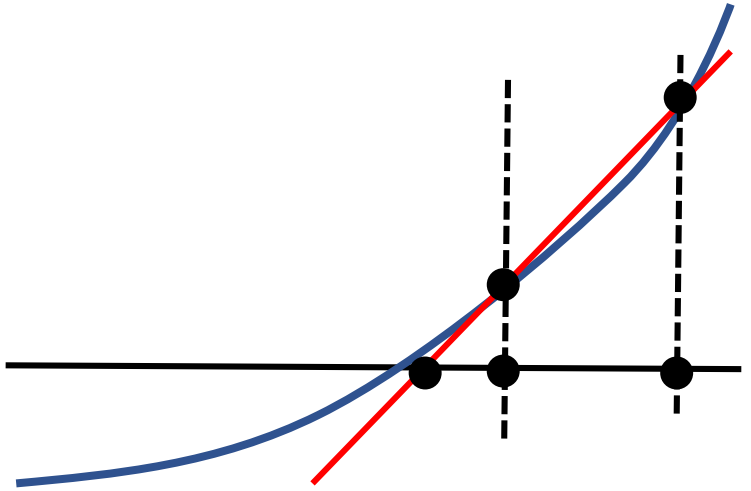
$$y - f\left(x^{(k)}\right) = m(x - x^{(k)})$$ %The line pssing through (x_1, y_1) with slope m is: $y - y_1 = m(x - x_1)$

- The next estimate $x^{(k+1)}$ is the intersection point of this secant line with x axis.
- It means $0 - f\left(x^{(k)}\right) = m(x^{(k+1)} - x^{(k)})$
- Next Estimate

$$\Rightarrow 0 - f\left(x^{(k)}\right) = \left(\frac{f\left(x^{(k-1)}\right) - f(x^{(k)})}{x^{(k-1)} - x^{(k)}}\right)(x^{(k+1)} - x^{(k)})$$

$$\Rightarrow \boldsymbol{x^{(k+1)} = x^{(k)} - \left(\frac{f(x^{(k)})(x^{(k-1)} - x^{(k)})}{f\left(x^{(k-1)}\right) - f(x^{(k)})}\right)}$$

- Desired tolerance: $\left|x^{(k+1)} - x^{(k)}\right| < \varepsilon$



- Newton's method iteratively updates x as follows:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

- Since, $f'(x^{(k)}) \approx \frac{f\left(x^{(k-1)}\right) - f(x^{(k)})}{(x^{(k-1)} - x^{(k)})}$, then
- $\boldsymbol{x^{(k+1)} = x^{(k)} - \left(\frac{f(x^{(k)})(x^{(k-1)} - x^{(k)})}{f\left(x^{(k-1)}\right) - f(x^{(k)})}\right)}$

- ***Finite difference approximation: a numerical technique used to approximate derivatives of functions.***
- ***Forward:*** $\boldsymbol{f'(x) \approx \frac{f(x+h) - f(x)}{h}}$
- ***Backward:*** $\boldsymbol{f'(x) \approx \frac{f(x) - f(x-h)}{h}}$
- ***Centered:*** $\boldsymbol{f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}}$